多智能体软件开发系统项目说明

一、项目概述

本项目构建了一个基于 LangGraph 的多智能体软件开发系统,通过多个分工明确的智能体(Agent)协作完成软件开发任务。系统采用"产品经理 → 架构师 → 开发者 → 测试工程师 → 代码审查员"的流水线协作模式,并内置反馈循环机制,当测试失败或代码审查不通过时,任务会自动回流到开发者节点进行修复,直至通过验收。

系统当前使用模拟 LLM(MockLLM)跑通完整流程,无需 API Key 即可演示多智能体协作原理;同时预留了真实模型接入接口,可通过环境变量一键切换到 OpenAI、DeepSeek、通义千问等真实大模型。系统提供 Web 界面,通过 SSE(Server-Sent Events)实时推送每个智能体的协作过程,实现协作流程的可视化展示。

二、技术方案

2.1 系统架构

系统采用分层架构,共分为四层:

- **LLM 抽象层**(`core/llm.py`):定义统一的 `BaseLLM` 接口,提供 `MockLLM`(模拟模型)和 `OpenAILLM`(真实模型)两种实现。通过环境变量 `LLM_PROVIDER` 切换,`OPENAI_BASE_URL` 支持配置兼容接口(可对接 DeepSeek、Qwen 等)。
- **Agent 角色层**(`agents/definitions.py`):定义五个智能体角色的系统提示词、图标、颜色等元数据,形成角色注册表。
- **LangGraph 编排层**(`core/graph.py`):使用 LangGraph 状态图定义智能体协作流程与条件路由,管理协作状态(任务、历史、当前角色)。
- **Web 服务层**(`web/server.py` + `static/`):基于 Flask 提供 REST API 与 SSE 事件流,前端通过 HTML/CSS/JS 实现协作过程可视化。

2.2 协作流程

系统采用有向状态图编排五个智能体的协作,核心流程如下:

```
产品经理 → 架构师 → 开发者 → 测试工程师 → 代码审查员
                             ↑                    │
                             │(测试失败)          │(审查不通过)
                             └──── 回流开发者 ┘
```

- **产品经理**:分析需求、拆解任务、定义验收标准。
- **架构师**:设计技术方案、划分模块、技术选型。

- **开发者**:实现功能代码、编写注释、处理边界情况。
- **测试工程师**:编写并运行单元测试、报告测试结果。
- **代码审查员**:审查代码质量、发现缺陷、给出通过/不通过结论。

条件路由逻辑:测试工程师输出包含"通过"则进入审查,否则回流开发者;审查员输出包含"通过"则结束流程,否则回流开发者修改。

2.3 关键技术选型

- **LangGraph**:用于构建有状态、可条件路由的多智能体协作图,支持反馈循环。
- **Flask + SSE**:提供轻量级 Web 服务与实时事件推送,实现协作过程可视化。
- **TypedDict 状态模式**:使用类型化字典定义图状态,确保状态在节点间正确传递与合并。

三、创新点

1. **角色化多智能体协作**:将软件开发流程抽象为五个职责分明的智能体角色,每个角色拥有独立的系统提示词与职责边界,模拟真实团队协作。

2. **内置反馈循环机制**:不同于简单的单向流水线,系统在测试与审查环节设置了条件路由,支持任务自动回流修复,更贴近真实开发中的迭代过程。

3. **模拟模型与真实模型无缝切换**:通过统一的 LLM 抽象层,系统可在无 API Key 的模拟模式下完整演示协作流程,也可一键切换到真实大模型,兼顾演示与生产需求。

4. **实时协作可视化**:通过 SSE 事件流将每个智能体的执行过程实时推送到 Web 前端,用户可直观看到任务在智能体之间的流转与各角色的产出。

5. **轻量可扩展**:核心代码仅约 900 行,模块划分清晰,新增智能体角色只需在角色注册表中添加定义,扩展成本低。

四、验证结果

系统已完成端到端验证,验证结果如下:

验证项	结果
核心图执行顺序	pm → architect → developer → tester → reviewer
协作历史记录	5 条(每个智能体各 1 条)
最终角色	reviewer(审查通过,流程正常结束)
Web 服务启动	端口 8081 正常监听
REST API	`/api/roles`、`/api/status`、`/api/run` 均正常响应

验证项	结果
前端资源	index.html、style.css、app.js 均返回 HTTP 200
SSE 事件流	正确推送 snapshot 及完整协作历史

验证表明,系统能够完整执行多智能体协作流程,Web 界面与后端 API 均工作正常,实现了"提交任务 → 多智能体协作 → 实时可视化 → 输出结果"的完整闭环。

五、当前局限

1. **模拟模型输出为模板化内容**:当前 MockLLM 基于规则生成确定性输出,内容较为模板化,仅用于演示流程,不具备真实推理能力。
2. **未接入真实大模型**:系统虽预留了 OpenAILLM 接口,但尚未配置真实 API Key 进行端到端验证,真实模型下的输出质量与稳定性有待测试。
3. **反馈循环逻辑依赖关键词匹配**:测试与审查的通过/不通过判定依赖输出中的关键词(如"通过"、" "),对真实模型的自由文本输出可能不够鲁棒。
4. **单任务串行执行**:当前系统一次只能运行一个任务,不支持多任务并发或任务队列。
5. **无持久化存储**:协作历史仅保存在内存中,服务重启后数据丢失,未接入数据库。
6. **前端功能较基础**:可视化界面展示了协作过程,但缺少代码文件落地、测试报告下载等进阶功能。

六、后续计划

1. **接入真实大模型**:配置 OpenAI/DeepSeek/Qwen API,验证真实模型下的协作质量,并针对自由文本输出优化通过/不通过判定逻辑。
2. **增强反馈循环**:引入更鲁棒的判定机制(如结构化输出、LLM 自评),支持多轮迭代修复与最大迭代次数控制。
3. **支持多任务并发**:引入任务队列与并发执行,支持批量提交多个开发任务。
4. **持久化与历史管理**:接入数据库存储协作历史与产出,支持任务回放与结果检索。
5. **代码落地与测试执行**:让开发者智能体真正生成可运行代码文件,测试智能体实际执行测试并生成报告,审查智能体基于真实代码审查。
6. **前端增强**:增加代码文件预览、测试报告下载、协作过程回放等交互功能。
7. **扩展智能体角色**:支持用户自定义角色,如增加"安全审查员"、"性能优化师"等,提升系统通用性。